

## W4D2 - Scheduling della CPU

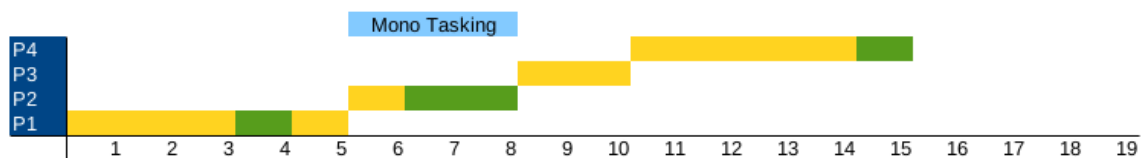
Oggi vedremo come le CPU pianificano, in un'ottica di ottimizzazione delle risorse, l'esecuzione dei processi.

I processi che prenderemo in esame hanno dei tempi di esecuzione e di attesa I/O come specificato nella tabella seguente:

| Processo | Exec      | Wait      | Exec (dopo il wait) |
|----------|-----------|-----------|---------------------|
| P1       | 3 secondi | 1 secondo | 1 secondo           |
| P2       | 1 secondo | 2 secondi |                     |
| P3       | 2 secondi |           |                     |
| P4       | 4 secondi | 1 secondo |                     |

### Mono Tasking

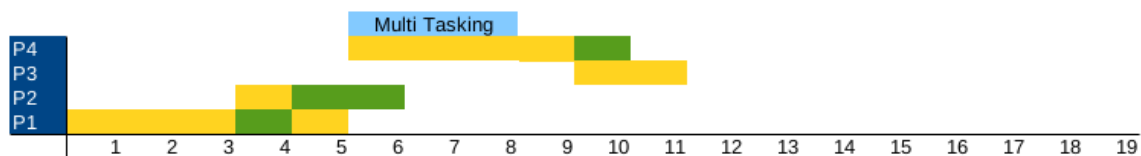
Uno scheduler Mono Tasking metterà in esecuzione i processi uno alla volta, senza curarsi dei tempi di attesa dovuti all'Input/Output da parte dell'utente, questo allungherà di molto i tempi di attesa per l'esecuzione dei processi partiti successivamente.



Come si vede nella tabella, ogni processo viene eseguito fino al suo completamento, attendendo per eventuali input da parte dell'utente, prima di passare al processo successivo.

### Multi Tasking

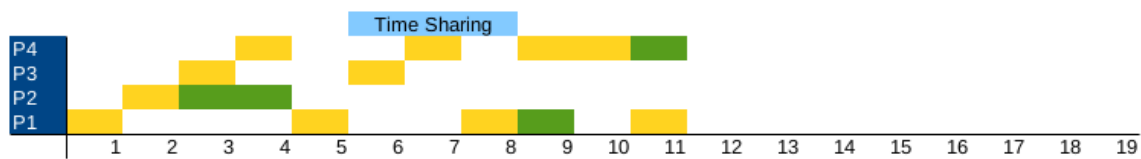
Al contrario del mono tasking visto precedentemente, il Multi Tasking è una tipologia di scheduler che permette di avviare in maniera concorrente i processi, per farlo utilizza i tempi di attesa di un processo per avviare quello successivo. Il beneficio più evidente è la diminuzione dei tempi di esecuzione totali in quanto non ci sono tempi morti tra un processo e l'altro.



In questa tipologia di scheduler i processi vengono avviati nel momento in cui il processo precedente passa in condizione di attesa, e ottimizzando l'avvio dei processi si ottiene un tempo totale molto ridotto rispetto all'esempio precedente.

## Time Sharing

Uno scheduler in Time Sharing, allocherà le risorse a più processi in base a degli slot di tempo predefiniti, in questo modo alla fine di ogni slot di tempo la CPU passerà al processo successivo per lo slot di tempo successivo senza tenere conto di eventuali attese per I/O e così via. Quest'allocazione temporizzata dà l'impressione di un sistema in grado di gestire in contemporanea più processi e porta il beneficio di un ridotto tempo di esecuzione totale come già visto per il multi tasking.



In quest'esempio vediamo come ogni processo ha un tempo predefinito di un secondo di esecuzione prima che la CPU passi al processo successivo, e ad ogni turno si va avanti fino a ripartire dal primo processo che può essere eseguito. Ad esempio il processo P2 ha un solo secondo di esecuzione e due secondi di attesa I/O, quindi al secondo 5 verrà eseguito direttamente il processo P3. Situazione analoga al secondo 8 in cui il processo P1 ha un secondo di attesa per cui la CPU passerà al primo processo attivo che è in questo caso il P5. Nel tempo di attesa di P5 verrà completato il P1 a cui restava ancora un secondo di esecuzione.

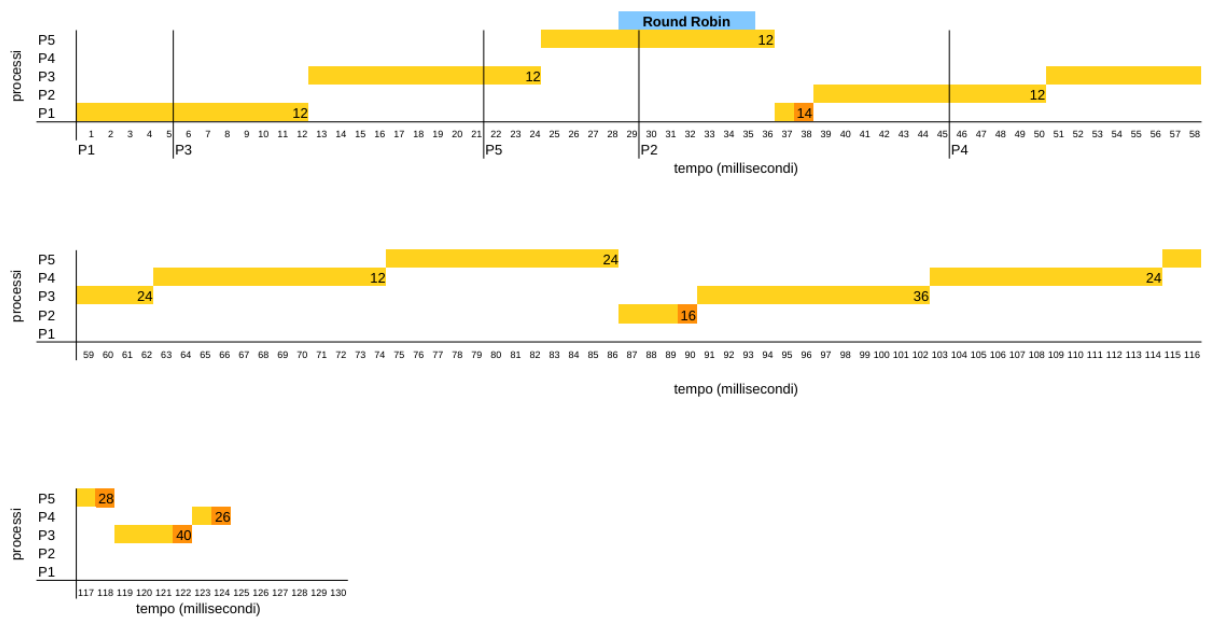
## Esercizio Facoltativo

### Round Robin

Un altro algoritmo che implementa il concetto di time sharing è il Round Robin, in cui la CPU alloca un tempo di esecuzione predefinito prima di passare al processo successivo, tenendo conto in questo caso del tempo di arrivo nella tabella di esecuzione, dando quindi priorità ai processi che sono già pronti per essere eseguiti.

Nell'esempio che segue, vedremo diversi processi con i tempi di arrivo ed esecuzione definiti in millisecondi nella tabella e un time-slice di 12 millisecondi.

| Processo | Tempo di arrivo ( $t_o$ ) | Tempo di esecuzione ( $T_x$ ) |
|----------|---------------------------|-------------------------------|
| P1       | 0                         | 14                            |
| P2       | 30                        | 16                            |
| P3       | 6                         | 40                            |
| P4       | 46                        | 26                            |
| P5       | 22                        | 28                            |



Come si evince dal grafico, il P1 è il primo ad essere eseguito, in ordine il P3 è il successivo ad essere pronto (già al millisecondo 6) e viene eseguito dopo il primo time-slice di 12ms. Il P1 termina la sua esecuzione al 38ms in quanto P4 non era ancora pronto e P2 viene dopo nella lista dei processi.

Questa è la tabella di esecuzione dei processi

| time slice | start time | end time | Process   |
|------------|------------|----------|-----------|
| 1          | 0          | 12       | P1        |
| 2          | 12         | 24       | P3        |
| 3          | 24         | 36       | P5        |
| 4          | 36         | 38       | P1 -> END |
| 5          | 38         | 50       | P2        |
| 6          | 50         | 62       | P3        |
| 7          | 62         | 74       | P4        |
| 8          | 74         | 86       | P5        |
| 9          | 86         | 90       | P2 -> END |
| 10         | 90         | 102      | P3        |
| 11         | 102        | 114      | P4        |
| 12         | 114        | 118      | P5 -> END |
| 13         | 118        | 122      | P3 -> END |
| 14         | 122        | 124      | P4 -> END |

In base ai dati presentati, con questo algoritmo si evince un tempo medio di attesa di **27.20 millisecondi** e un attesa di **26.87 millisecondi** affinché un processo torni ad usufruire delle risorse per un altro time-slice.

| Processo         | wait time      | turnaround     |
|------------------|----------------|----------------|
| P1               | 0ms            | 24ms           |
| P2               | 38ms           | 36ms           |
| P3               | 12ms           | 23ms (avg)     |
| P4               | 62ms           | 18ms (avg)     |
| P5               | 24ms           | 33ms (avg)     |
| <b>TOTAL AVG</b> | <b>27.20ms</b> | <b>26.87ms</b> |

Alla prossima.

A stylized handwritten signature in black ink, featuring a large 'D' and 'M' with a cursive 'ain' in between, all under a long horizontal flourish.